Pin and Replay, or Refuse: Exact Incremental Re-annotation Across Reference Database Releases

Joshua Terranova
Thomas Lord Department of Computer Science
University of Southern California

Jiapeng Zhang
Thomas Lord Department of Computer Science
University of Southern California

Abstract

Biologists annotate genomes by searching their proteins against reference databases such as Pfam, which publish a new release every few weeks to months. Each release forces a full re-search, although most of the database has not changed: 86.6% of Pfam's entries matched between releases 38.1 and 38.2 after excluding build metadata and comments. Reusing the old results is hard, because the search programs are fixed binaries whose printed statistics depend on the size of the whole database. We present TARE (Tool Annotation Reuse, Exact), a wrapper that reruns an unmodified program only on the entries that can matter, with the program's own database-size setting pinned to the new release, so that the program itself prints the numbers a full run would print. TARE then tests the replay by adding back entries it left out, and refuses, returning an ordinary full run, if any line changes. Under the contract, every output line except comment headers is byte-identical to a full rerun; the contract assumptions are tested on each update, not proved. In the all-record timing reported here, TARE also reruns the program in full on 299 sampled records drawn from all query records, so a hidden dependency that affects at least 1% of that population ships with probability at most 5%, assuming a valid audit oracle.

On Pfam 38.1→38.2, TARE is 3.47× faster than a full rerun under that all-record bounded audit (median over 8 genomes after excluding 1 wall-quality flagged genome; 95% CI 3.30–3.81). Historical candidate-population timings on the same nine genomes were 4.24× (Bernoulli audit; 95% CI 4.17–4.68) and 3.49× (bounded s=299; 95% CI 3.32–4.10). Cost of checking versus unchecked replay is 1.58× (C/B over the same unflagged set). Across 715,704 output lines from HMMER and Infernal, not one differed from a full run. TARE refused DIAMOND, whose first filter reads bytes of the neighboring database entry, and BLAST+, whose unpinned probe rows fit none of TARE's column rules.

1 Introduction

When a laboratory sequences a genome, it obtains thousands of protein sequences with no labels. To learn what each protein does, researchers compare it against *reference databases*: curated collections of known protein families, such as Pfam [20], UniProt [28], and InterPro [21]. A program such as HMMER [12] scores every protein against every family and reports the good matches, called *hits*. This step, called *annotation*, underlies much of modern biology, from comparative genomics to the sequence alignments used by structure predictors such as AlphaFold [16].

Reference databases keep improving. UniProt and InterPro publish a new version, called a *release*, roughly every eight weeks [21, 28], and Pfam published eight releases between May 2024 and June 2026 [23]. Each release adds and revises families, so annotations computed against an old release go out of date. The annotation programs keep no memory of earlier runs, so the only way to update is to compare every protein against the entire new database again. Most of that work recomputes answers that did not change: between Pfam 38.1 and 38.2, 86.6% of the 30,134 families were identical apart from build-date, build-command, and comment lines (Figure 1). At scale this repeated cost shapes practice. NCBI re-annotates its RefSeq collection on a rolling schedule because redoing all of it at once "requires securing at once a significant amount of computing resources" [18], and JGI states that at over 65 billion genes "it is impossible to upgrade all genome and metagenome annotations to the latest version" [5].

Why reuse is hard. It might seem easy to search only the new and edited families and keep the old results for the rest. It is not, for three reasons. First, the results are *coupled*: the statistic HMMER prints for each hit, its E-value, is scaled by the total number of families in the database, much as a grade curved to class size changes for every student when new students enroll. Adding families therefore changes the printed numbers of hits on families that did not change. Rerunning HMMER on exactly the right families, without accounting

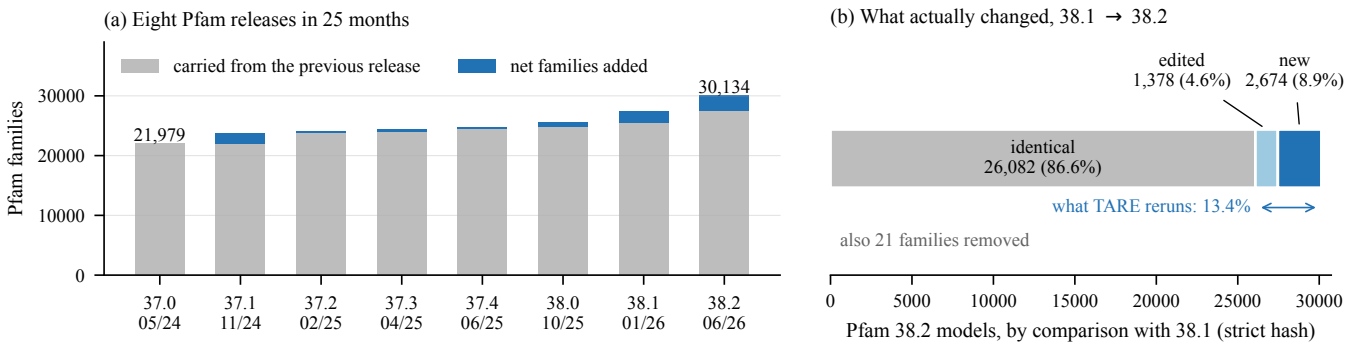


Figure 1: **Reference databases change often, but usually only in part.** (a) Pfam grew from 21,979 to 30,134 families over eight releases; blue segments show the net families each release added. (b) Comparing every model of Pfam 38.2 with 38.1, ignoring build-date, build-command, and comment lines: 86.6% are identical. Only 13.4% (new or edited) can produce new results, yet a conventional update recomputes 100%. Blue marks what TARE reruns. Panel (b) is the pair we time; Figure 6 measures the six neighboring releases the same way (median changed fraction 12.2%).

for this, changes 9,083 of 9,094 output lines for one bacterial genome. Second, the programs are unmodified binaries maintained by others; we cannot change their code. Third, an approximately correct answer is not good enough: pipelines that track provenance must reproduce a full run exactly, because "pipeline differences can lead to annotation discrepancies" [5].

Existing tools fall short in two ways. General-purpose caches, such as build systems and workflow managers [7, 11, 19, 26, 30], remember whole commands keyed by the bytes they read. A new release changes the database file, so they rerun the search in full. Specialized incremental search tools such as iBLAST and iSeqSearch [9, 31] are written by hand for one program and adjust the printed numbers after the fact. In the one configuration we ran, only 1.3% of the rows that iSeqSearch shared with a full run were byte-identical, and it reported 133 hits to sequences that the new release had deleted. *How can we reuse earlier annotations across a database release, without modifying the programs, and either return exactly what a full rerun would print or knowingly fall back to one?*

The key idea in plain terms

In the programs we studied, entries in the database influence each other's printed results through only a few global numbers, chiefly the database size, and the programs already accept those numbers as settings. If we tell the program the new release's size up front (*pin* it), it scores each family on its own merits. Then we can rerun it on a small subset of the database and it prints exactly what a full run would print, digit for digit, because the program, not our wrapper, computes every number. This is what *taring* a scale does: fix the baseline first, then weigh only what is new. When a program breaks this pattern, we try to detect it and fall back to a full run.

Pinning a global size is not new in itself. Parallel search tools split a database across machines and fix the database size so that every piece prints consistent E-values [8], and HMMER's `-Z` option exists partly for that purpose [12]. What is missing is a way to use it across releases, for a binary we do not control, without trusting that the binary has no other coupling. TARE supplies that: it finds which entries changed (Section 3), reruns each protein only against its few candidate families with the size pinned (Section 4.5), and before returning anything adds back entries it left out and refuses if any line changes (Section 4.6). Every refusal falls back to an ordinary full run, so users always receive the program's own output. Worked numbers for Pfam 38.1→38.2: pin `-Z 30134` and, on one genome, replay 3,719 candidate families rather than all 30,134 (Figure 2).

We evaluate TARE on four programs: HMMER and Infernal, whose evaluated shipped runs in supported configurations matched full runs, and DIAMOND and BLAST+, which it refuses. Infernal updates ran about twice as fast as a full rerun, with a reduced check on long-DNA inputs (Section 6.3). Our contributions are:

1. **Exact replay across releases with check-or-refuse.** Release-diff selection and candidate-only, batched, pinned replay that makes an unmodified program print byte-identical output, guarded by a probe-based gate, a fallback when replay cannot pay, and a check whose audit bounds the chance of shipping a wrong update (Section 4).
2. **A root cause for a refusal.** Investigating a failure of our first check revealed why DIAMOND cannot be replayed exactly: its first filter reads bytes belonging to the neighboring database entry, so its results depend on database order (Section 6.4).
3. **A pre-registered evaluation.** Exactness over 715,704 output lines; Pfam 38.1→38.2 all-record bounded-audit median 3.47× (95% CI 3.30–3.81; 8 unflagged genomes); historical candidate-population medians 4.24× (Bernoulli) and 3.49× (bounded s=299); known-bad controls that must refuse; and comparisons with caches and approximate incremental search, with designs locked before the

Table 1: Terms used in this paper.

Term	Meaning
Record	One query item, here one protein (or one DNA sequence for Infernal).
Entry	One family model in the reference (a profile HMM in Pfam).
Release	A new version of the reference.
Pin	Fixing a global setting (e.g., Z) to the new release's value.
Replay	Rerunning the unmodified program on a subset of the reference.

measured runs (Section 6).

2 Background

2.1 How Annotation Works

An annotation program takes two inputs: a set of *records*, such as the thousands of proteins of a bacterial genome, and a *reference* of *entries*, such as Pfam's 30,134 protein families (Table 1). Each Pfam family is stored as a statistical profile, a profile hidden Markov model, that summarizes what members of that family look like. HMMER's `hmmscan` compares every protein with every family, computes a score, and writes one line per reported hit, containing the names of the protein and family, the score, and an E-value. The E-value estimates how many equally good matches would occur by chance in a database of this size, and HMMER computes it as a probability multiplied by Z, the number of families in the database [12]. With Pfam's curated per-family score thresholds (the `--cut_ga` option), whether a hit is reported depends only on its score against that family's threshold [20].

2.2 Why Naive Reuse Fails

Suppose a laboratory annotated a genome with Pfam 38.1 and wants Pfam 38.2 results. The obvious shortcut is to search only the 4,052 families that are new or edited and keep the cached lines for the other 26,082. Nearly every kept line is then wrong, because its E-value was multiplied by 38.1's Z (27,481) instead of 38.2's (30,134) (Figure 2). Rerunning HMMER on exactly the right families does not help either, because the program then uses the subset's size as Z: on one genome, such a rerun differed from a full run on 9,083 of 9,094 lines, and the other 11 lines matched. Multiplying the cached E-values by $Z_{\text{new}}/Z_{\text{old}}$ gets closer but still fails, because printed numbers are rounded, and the rounding of a rescaled number differs from the program's rounding of its own internal value.

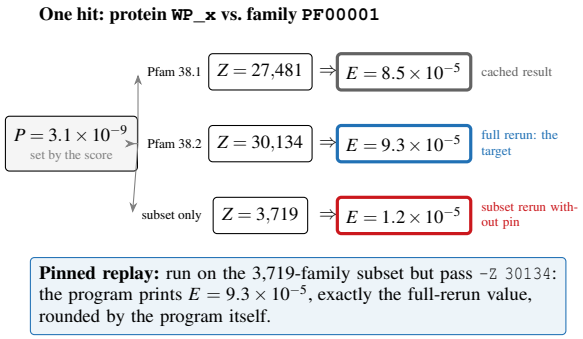


Figure 2: **Why old results cannot simply be reused.** An E-value is a probability P (fixed by the score) times the database size Z. The same hit prints a different E-value in each release, and a different one again if the program is rerun on a subset. Pinning Z makes the subset run print the full-run value. (Illustrative P; Z values are Pfam 38.1, 38.2, and one proteome's candidate subset.)

3 Problem Model

Program and release. A black-box program P maps records R, reference E, and options o to a tabular body (headers ignored). Each entry of the ordered list E has a strict hash $h(e)$ (SHA-256 after dropping caller-declared volatile tags). A release $E \to E'$ yields stable $S = \{e \in E' : h(e) \in h(E)\}$, changed $\Delta = E' \setminus S$, and $f_\Delta = |\Delta|/|E'|$. For each pair (r,e), let $f(r,e,o,g(E'))$ be its possibly empty sequence of emitted body rows, including bytes and multiplicities, where $g(E')$ collects reference-wide quantities (entry count, length, ...). Exact re-annotation reproduces body$(P(R,E',o))$ under the conditions below; an observed check failure returns the full run instead.

Coupling taxonomy. **C0** (entry-local): f independent of g and of every $e' \neq e$ (Infernal's no-clan variant) $\to$ replay. **C1** (reference-size): f depends on E' only through a normalizer in g that P exposes or TARE rescales (HMMER `-Z`) $\to$ pin, or refuse if uncovered. **C2** (neighbor/order): the line for (r,e) depends on neighbors of e or on input order (DIAMOND) $\to$ refuse. **C3** (cross-record global, e.g. FDR) and **C4** (nondeterministic) $\to$ refuse when observed. Refusal rules R1–R6 form one surface: R1 no contract; R2 size-dependent, no pin; R3 in-situ pin check fails; R4 non-candidate pair in replay; R5 pad changes a line; R6 audit differs. C0/C1 replay; C2–C4 refuse (R1–R3 on the probe, R4–R6 on the update). Side-file rewrites and invalidate-groups are *declarations*; they do not establish entry locality for competing groups.

Exactness (C0/C1). Assumptions: (D) **determinism**—fixed (r,e,o,g) emits the same rows; (P) **complete pin**—pinned options fix every reference-wide quantity affecting row existence, multiplicity, or bytes to its full-release value,

including quantities not observed by the fitter, without changing other output-relevant behavior; (K) **attribution**—each row occurrence is attributable to its record and entry, preserving repeated occurrences; (O) **exact assembly**—the merge reconstructs stock ordering and formatting from these rows and the original record/reference order; (C) **candidate completeness**—every entry producing a full-release row for r belongs to its candidate set C_r; (I) **record-subset invariance**—with reference and options fixed, running on a subset of records produces exactly the projection of the full-record body onto that subset.

Proposition 1 (Exact merge)**.** Assume C0 or C1 and (D), (P), (K), (O), (C), and (I). Running P on each r against C_r with g pinned, then applying the merge in (O), yields body$(P(R,E',o))$.

Proof. By (I), isolating r preserves its full-reference rows. Under C0/C1 and (P), restricting the reference to C_r preserves the emitted rows for each retained pair, including empty sequences and multiplicities; (D) excludes execution drift. By (C), every omitted pair emits no full-release row. Replay therefore produces exactly the full-release row occurrences; (K) preserves attribution and (O) reconstructs the full body. □

The probe (R1), pin check (R3), pads (R5), and non-candidate tripwire (R4) test these assumptions; they do not prove C0/C1 on the full input. Not covered: C2–C4; residual couplings below κ or that evade pads and the audit; limitation F6 (coupling outside the closed family that still looks locally consistent).

Audit bound. Let A be the stated audit population and fix the affected records independently of the audit draw. Under (I), sample-only stock is a valid oracle. If at least a fraction κ of A differs, a uniform sample of $s \le |A|$ records without replacement misses them with probability at most $(1-\kappa)^s$. The conservative with-replacement sizing rule is

$$s = \left\lceil \ln \varepsilon / \ln(1-\kappa) \right\rceil. \tag{1}$$

For $\varepsilon = 0.05$, $\kappa = 0.01$, $s = 299$; if $|A| < s$, audit all of A. The bound requires that the stated audit executes with a valid oracle. For the historical bounded timings, A contains only candidate-bearing records; the all-record policy uses $A = R$ and requires its own timing evidence. In the historical adversarial evaluation, 3 of 400 trials shipped when at least 1% of its audit population was affected.

What the audit does *not* cover

It is not a 5% per-line error rate, nor a proof that the program is entry-local. Couplings that affect fewer than κ of records can still ship; so can structured faults that evade the pads and the sample (limitation F6). C2–C4 are refused when observed (R1–R6), not "covered" by s=299.

Cost model. With stock time T_{stock},

$$T_{TARE} \approx a\, f_\Delta\, T_{\text{stock}} + c\, T_{\text{stock}}, \tag{2}$$

where a scales selection and c collects replay, check, audit, and parsing. Fallback when $f_\Delta > (1-c)/a$; default threshold 0.80 with no timing history. Programs whose rows factor as $f(r,e,o,g(E))$ and satisfy the exact-merge assumptions fit; C2–C4 remain refuse classes. TARE never reads the program's source at run time.

4 Design

TARE updates a cached annotation in six stages (Figure 3). The first three decide *whether* exact replay is possible and *how*; the last three perform it and check it. Any failed decision or check leads to a full stock run. Passing these finite checks supports the contract on that run; it does not prove it.

4.1 Stage 1: Entry Diff

TARE splits both releases into entries and computes a *strict hash* of each: SHA-256 over the entry's full text, including its name, accession, description, length, Pfam's curated thresholds (GA, TC, NC), and every model line, after dropping only lines tagged DATE (build date), BM and SM (the commands that built and calibrated the model), or COM (comments). A renamed or re-thresholded family therefore counts as changed. Entries of the new release whose hash appears in the old one are stable; all others are changed. A cache keyed on the whole database file would miss on every entry after a release; a cache keyed on entry hashes keeps every stable entry, 86.6% of Pfam for 38.1→38.2.

4.2 Stage 2: Contract Inference

Without reading the program's code, TARE probes a sample of records three times—whole reference and each half—and aligns lines by record and entry. If every whole-run line appears in exactly one half, the output *decomposes* over entries; a whole-only line means unreplayable interaction (refuse). A half-only line is allowed only when the user declares competing entries (Section 4.3). For each numeric column it tests identity or scaling with entry count, total length, or hits per record; a fit within printed precision is required, else refuse. Ranks and labels are left to the program. The resulting *contract* is computed once per command and reused across releases.

4.3 Stage 3: The Coupling Gate

The gate turns the contract into a decision. Users may add declarations, which are data, never program names.

Size dependence → pin. If a column scales with the reference size, TARE needs a program option that sets that size.

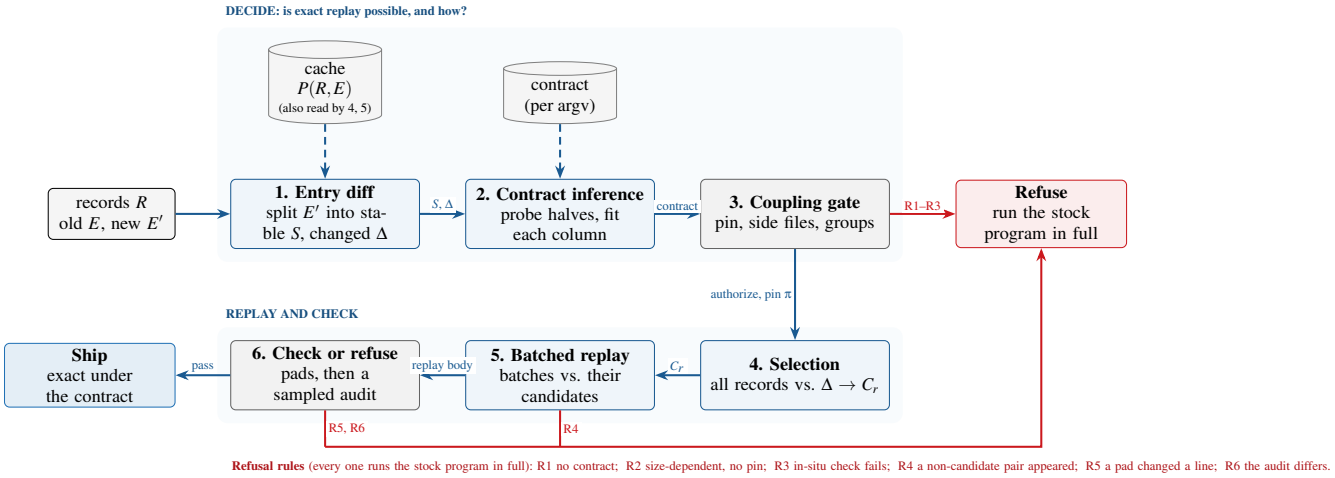


Figure 3: **How TARE updates an annotation.** Blue boxes do work; gray boxes are decision points. The top row decides whether the program's coupling can be neutralized; the bottom row reruns the unmodified program on candidates only and checks the result. Dashed arrows read stored state. Every refusal (red) leads to a full stock run, so a user always receives the program's own output. A shipped replay is exact under the contract; finite checks do not prove the contract.

Users declare it (`--pin '-Z {entry_count}'`), and every experiment in this paper used a declared pin. TARE can also search for one, by reading the program's help text and trying each numeric option until one moves the column by exactly the predicted ratio; we tried this search only on HMMER and DIAMOND, and we have not evaluated its risks, such as an option like a random seed moving a column by coincidence. With the option set to the new release's value, the program prints on a subset the numbers it would print on the whole.

No size dependence → no pin. If every column fits identity, TARE replays without a pin. This is the case for Infernal under Rfam's recommended command, which derives its search size from the query sequence instead.

Side files and competing entries. Rfam's recommended command passes a "clan" file listing families that compete. On a subset, TARE rewrites that file to the families present, because the program rejects names it cannot find. If any member of a declared group changes, TARE re-searches the whole group, because a change to one member can reveal a hit of another that the cache never recorded.

The gate applies R1–R6 from Section 3 (R1–R3 on the probe; R4–R6 on the update). Every refusal runs the program in full; REFUSE_PIN (R2/R3) differs from REFUSE only in reporting that a working pin could enable replay. Before replaying, the gate also falls back when f_Δ exceeds the break-even point $(1-c)/a$ of Equation (2) (default 0.80 with no timing history).

4.4 Stage 4: Selection

Changed entries may produce hits the cache has never seen. TARE runs the program on all records against the changed entries Δ alone, with the pin set. A record's *candidates* are its cached hits on stable entries plus its new hits:

$$C_r = \{e \in S : (r,e) \in P(R,E)\} \cup \{e : (r,e) \in P_\pi(R,\Delta)\}. \quad (3)$$

Selection compares every record with every changed entry, so its cost is proportional to the changed fraction of the reference. It is the one cost TARE cannot avoid (Section 4.7). Candidate completeness for the merge is $\text{Hits}(r,E_{\text{new}}) \subseteq \text{Cached}(r) \cup \text{Discovered}(r)$, which holds here because Z is pinned and reporting thresholds are per family (`--cut_ga`); without those conditions an unchanged pair that was previously below a global threshold can become reportable and be missing from both sets.

4.5 Stage 5: Batched Exact Replay

Cached lines cannot be returned as they are: their numbers used the old release. TARE reruns the program on the candidates with the pin set so the program reprints. A naive all-candidates universe wastes work—on one genome the union covers 3,719 families while the mean protein has two—so proteins are batched against only their own candidates and stitched in record order, choosing the grouping that minimizes

$$T_{\text{replay}} = \alpha k + \beta \sum_b |U_b| + \gamma \sum_b |B_b|\,|U_b|, \quad (4)$$

with k batches, $|B_b|/|U_b|$ proteins/families in batch b, and α, β, γ start, prepare, and compare costs. HMMER's per-protein blocks depend only on that protein's hits. Tripwire R4: a pair outside C_r refuses.

4.6 Stage 6: Check or Refuse

The check falsifies C0/C1 on the run: replay with *pads* (left-out families added back). The *neighbor pad* adds file-order neighbors of every candidate (R5; catches C2 as in DIAMOND, Section 6.4). *Random pads* add three random left-out samples, each as large as the candidate set, on s=50 proteins. If a dependency affects k of n proteins and a pad hits the responsible family with probability q,

$$P_{\text{miss}} \approx h + (1-h)(1-q)^3, \qquad h = \binom{n-k}{s} \Big/ \binom{n}{s}. \quad (5)$$

Pads test C0/C1; the audit bound is R6 with s from Equation (1). The historical bounded policy samples only records that have at least one replay candidate; extending the bound to all input records (including empty candidate sets) is in progress and is not claimed for the 3.49× timing. The audit oracle reruns stock on the sampled records alone; we treat record-subset invariance of that oracle (sample-only stock equals the projection of full-query stock) as an assumption whose test is in progress. Runs that predate that bounded policy used an earlier audit (p=0.02, floor 20). An optional first-use mode compares the first two updates to a full run.

4.7 Cost Model

Equation (2) in Section 3 is the working model: selection scales with f_Δ, and c collects replay, check, audit, and parsing. The speedup $1/(af_\Delta + c)$ never exceeds $1/f_\Delta$ or $1/c$. The first run also fills the cache (a full run plus contract probes); with fill time F and update time U, TARE becomes cheaper than repeated full runs after $n \geq (F - T_{\text{stock}})/(T_{\text{stock}} - U)$ releases.

5 Implementation

TARE's core is 12,555 lines of Python, with 358 test functions. Its core contains no program names: small parsers read profile databases, covariance-model databases, FASTA files, and tabular outputs, and users declare pins, side files, groups, and reporting thresholds as data. The cache is a SQLite file keyed by entry hash and record. A typical invocation wraps the normal command shown below. The cache is namespaced by a hash of the command line with the reference path removed, so changing any option, including the thread count, starts a new cache and a new contract. This is conservative: repeated stock runs were byte-identical in every timed experiment. The namespace also includes a SHA-256 of the program binary, so upgrading a program in place starts a new cache.

```
tare run --strategy reference --pin '-Z {entry_count}' \
  --cache cache.sqlite -- hmmscan --cut_ga --noali --cpu 4 \
  --tblout {output:tblout} --domtblout {output:domtblout} \
  {reference} {input}
```

6 Evaluation

We ask four questions. **Q1:** Is TARE's output identical to a full run? **Q2:** How much faster is an update, and does the cost model predict it? **Q3:** Does TARE refuse when replay would be wrong? **Q4:** How does it compare with existing tools?

Method. Every experiment followed the same protocol, designed so that a failure means the method failed rather than that the test was blind. We first read the program's source for behaviors that could make a comparison fail for unrelated reasons, then ran a pilot on separate data, then committed the design and pass/fail rules to version control before the measured run. Each test included a case that must pass and a case that must fail, and a minimum number of output lines below which a result counts as inconclusive. Measured runs used genomes drawn by a fixed random seed and never used before. One rule changed after its pre-registration, during a measured run; we disclosed the change and reran every control (Section 6.4).

Setup. HMMER 3.4 (`hmmscan`) with Pfam 38.1 and 38.2; Infernal 1.1.5 (`cmscan`) with Rfam 15.0 and 15.1; DIAMOND 2.2.5 and BLAST+ 2.14.1 with Swiss-Prot 2026_01 and 2026_03; queries are bacterial genomes from NCBI RefSeq. Server timings use a dedicated AMD EPYC 7542 node; laptop timings use an Apple laptop shared with other work and are indicative only.

6.1 Metrics

Exactness. We separate three statements: (i) *exact under the contract*—Proposition 1 under C0/C1 and (D)–(O), (C), and (I); (ii) *byte agreement on evaluated runs*—a run is EXACT in tables if its body lines equal the stock run's as byte sequences; (iii) *bounded detection* for a stated class of faults outside the contract (pads and the audit). Lines within a run share a pin and a reference, so they are not independent observations. We report compared-line counts and observed differences without inferring a per-line error probability. **Speedup.** For genome g with r alternating runs per arm, S_g is the median stock wall time divided by the median TARE wall time, both measured around the whole process. Across genomes we report the median of S_g with a 95% percentile bootstrap interval, the geometric mean, and the range. Differing lines are classified as whitespace-only, value-only, missing, extra, or reordered. Safety outcomes are EXACT, SAFE-REFUSE, or UNSAFE (any UNSAFE fails the claim). Wall time T is monotonic elapsed time of one complete invocation excluding `hmmpress`. Baseline overhead uses paired $\rho = T^{\text{tool}}/T^{\text{stock}}$ with a 95% t-interval.

6.2 Q1: Is the Output Exact?

Across the eight experiments in Table 2, not one of 715,704 compared lines differed from the stock run. This is observed

Table 2: Exactness experiments. “Lines”: output lines compared byte for byte with a stock run on the new release (HMMER: hit plus domain table). “Differ”: lines that differed. “Unpinned”: the same replay without the pin. “Replay” runs the replay engine alone; “tool” runs TARE end to end, with batching and the check.

Experiment	Lines	Differ	Unpinned differ
HMMER replay, 300 proteins	1,079	0	534 of 534
HMMER replay, 3 genomes	51,406	0	26,094 of 26,127
HMMER tool, 2 genomes (laptop)	35,434	0	not run
HMMER tool, 9 genomes (server)	155,577	0	not run
HMMER tool, 48 diverse genomes	463,225	0	not run
Infernal replay, 10 genomes	4,217	0	no pin needed
Infernal tool, 1 family removed	459	0	no pin needed
Infernal tool, 10 new genomes	4,307	0	no pin needed
Total	715,704	0	

agreement on those runs, not a probability guarantee for other inputs. Without the pin, nearly every line differed: 26,094 of 26,127 hit lines on three genomes ($d = 99.87\%$) and all 534 on 300 proteins. In these experiments, the pin was necessary; with `--cut_ga`, pinned replay matched stock.

Diverse genomes and needless refusals. On 53 further genomes (14 archaea, 30 bacteria, 2 eukaryotes, and 7 metagenome-assembled genomes), TARE shipped 48 updates, all byte-identical, and produced no unsafe result. It refused the other 5 although their replays matched stock: 3 because two domain-table rows of one protein shared the key (protein, family, E-value), and 2 because a text column held some values that parse as numbers. The *Drosophila* proteome, with 96 proteins of more than 20 domains, was one of the latter; three further eukaryotes did not finish within 12 hours and one bacterial job crashed on a storage error, so neither is counted. The duplicate-key rule also refused one of the ten server genomes below. These refusals cost speed, not correctness. After the measured runs we fixed both rules: repeated keys are told apart by their order, and a column that is mostly text is passed through. All six needlessly refused HMMER genomes, including the server genome and *Drosophila*, then shipped byte-identical updates (a follow-up, not part of any locked draw).

Where exactness ends. When HMMER reports hits by E-value threshold instead of `--cut_ga`, the replay matched on 300 proteins but failed on 1,000, differing on 3 hit lines and 5 domain lines. Hits near the threshold move in or out of a protein’s list, which changes that protein’s statistics; this mode is not entry-local and we do not claim it. The early pass on 300 proteins is why we require a minimum number of lines before accepting any result.

A second program. Infernal, which searches RNA families, behaved as its source code predicted: under Rfam’s recommended command every numeric column fits identity, so no pin is needed. Replaying each genome’s hits was

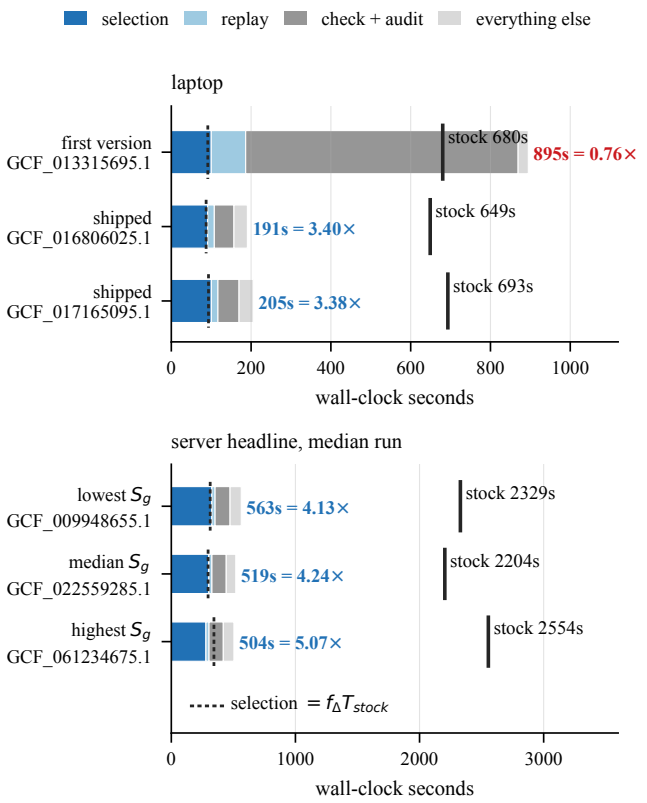


Figure 4: **Where the time goes.** Each bar is one update run, split into four stages; the black tick is a stock run on the same genome. Top (laptop, indicative): the first version, the median of three alternating runs, spent 668 s in its safety check and was slower than stock; batching and sampling (two fresh genomes) cut the update to under a third of a stock run. Bottom (historical Bernoulli audit): the median-time update of the genomes with the lowest, median, and highest speedup. Dotted ticks: the cost model’s prediction for selection.

byte-identical on ten genomes (4,217 lines) with and without clan competition. In that command, overlapping hits of competing families are marked rather than deleted, so removing a family cannot reveal a hidden hit; removing the family `GlmZ_SraJ` from one genome’s reference was exact through the shipped tool (459 lines). With the non-default option `--oskip`, which deletes marked overlaps, the same removal was refused, because fourteen overlap marks depended on families that had no remaining line. Through the shipped tool on ten further genomes (Rfam 15.0→15.1, three alternating runs per arm), every one of the 60 updates was byte-identical to its paired stock run in both variants, with 4,307 body lines per variant against a pre-registered floor of 2,000.

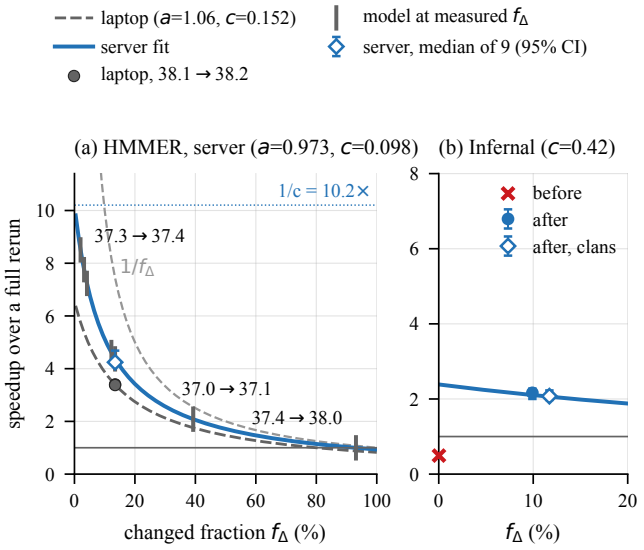


Figure 5: **The cost model across release sizes.** Speedup $1/(af_\Delta + c)$. (a) HMMER: the server fit from the nine historical Bernoulli-audit genomes ($a = 0.973$, $c = 0.098$, solid) with that policy's median and 95% CI (open diamond); the laptop fit ($a = 1.06$, $c = 0.152$, dashed) and its two updates (filled circles); the server model at the measured f_Δ of the other Pfam pairs (ticks); the bound $1/f_\Delta$ (dashed gray) and the server ceiling $1/c$ (dotted). (b) Infernal, with its own larger overhead: the first version (cross) and the shipped tool, medians of ten genomes with 95% bootstrap CIs.

6.3 Q2: How Much Faster?

Figure 4 shows that the replay and check designs decide whether TARE is faster than a full run. Our first version replayed every protein against all candidates and checked the result with four more full-size replays. On the laptop, with three alternating runs per arm, it took a median of 895 s against a median stock time of 680 s ($S_g = 0.76\times$; CV 3.7% for TARE and 1.7% for stock); the check alone took a median of 668 s. Batching made the replay 5.2× cheaper and sampling made the check 17.9–19.3× cheaper (ratios of the first version's median check time to each shipped run's). On two fresh genomes of 4,583 and 5,019 proteins, TARE replayed in 107 and 111 batches and took 191 s and 205 s against stock runs of 649 s and 693 s: $S_g = 3.40\times$ and $3.38\times$ (geometric mean $3.39\times$), with byte-identical output. These are single runs on a shared machine, so we report no confidence interval for them; the server headline repeats each arm three times.

The cost model. With $f_\Delta = 0.134$, the selection term predicts 87 s and 93 s on those genomes; selection measured 91 s and 100 s, a relative error of 4.3% and 7.1%. The remaining costs, as a fraction of each stock run, average $c = 0.152$ (arithmetic mean of 0.153 and 0.151), so even a release that changed nothing could be at most 6.6× faster with today's overheads (Figure 5); the check, audit, and parsing are therefore the next targets. On the server, the same fit over the nine headline genomes gives $a = 0.973$ and $c = 0.098$, a ceiling of 10.2×, and at headline f_Δ=0.134 predicts 4.37× versus the measured bounded-audit median 3.49×. The fit uses the lighter Bernoulli policy; it does not isolate the causes of the gap to the bounded policy (s=299). The laptop cache fill took 871 s and 852 s (single runs), 1.2–1.3× a stock run, so TARE pays for itself from the first update (projected from these indicative times).

Other releases. Strict hashes of all seven consecutive Pfam pairs (Figure 6) put the changed fraction between 2.0% and 93.1%. At those measured values the server model predicts 4.6–8.5× for the four other pairs that changed at most 12.2%, 2.1× for 37.0→37.1 (39.3%), and 1.00× for 37.4→38.0, which rewrote 93.1% of Pfam's models. That release was a rebuild, not churn: 22,925 of its 22,965 edited models carry a new accession version, which `hmmscan` prints, and 19,976 have different model parameters. For such a release replay cannot pay, so the gate falls back to a full run (Section 4.3): with the server fit the break-even fraction is 0.93, and on a 40-model subset of this release ($f_\Delta = 0.975$) the default threshold fell back and the output matched stock. The headline times the 38.1→38.2 pair. On the measured release chain (three fresh genomes, seed 20261222, exclusive EPYC 7542, r=3 per step), a cache filled at Pfam 37.3 and updated through 37.4, 38.0, 38.1, and 38.2: two of three genomes shipped exact (byte-identical) at every step. Per-step S_g was 7.08×/8.92× at 37.4, 0.91×/0.68× at 38.0, 4.29×/4.54× at 38.1, and 4.01×/5.35× at 38.2. Cumulative wall including the fill, $C_{\text{stock}}/C_{TARE}$ after the last step, was 1.66 and 1.71 (after 37.4 alone: 1.31 and 1.46). The 38.0 step is the case the break-even fallback is for: this run used frozen code from before that fallback, so it shows the cost without it. The third genome was refused at the fill (37.3) for a duplicate row key, fell back to a full scan, and shipped no chained output—the same needless-refusal class a later row-key fix addressed; the measured chain used the pre-fix code and counts a scored FAIL.

Server headline (all-record). On a dedicated node, across the same nine genomes with three alternating stock and TARE runs each under the locked all-record bounded audit (s=299), the median of S_g over 8 non-FLAGGED genomes is 3.47× (95% genome-cluster bootstrap CI 3.30–3.81; geometric mean 3.51×; range 3.28–3.96×). 1 genome was wall-quality FLAGGED and excluded from that median (GCF_049949335.1; all-genome sensitivity 3.44×). Every scored cell was body-equal to stock (466,725 compared tblout+domtblout lines). Cost of checking versus unchecked planner replay is 1.58× (C/B; range 1.53–1.71× on unflagged genomes).

Historical candidate-population timings. On the same nine genomes, the earlier candidate-population bounded audit (s=299) gave median S_g=3.49× (95% CI 3.32–4.10),

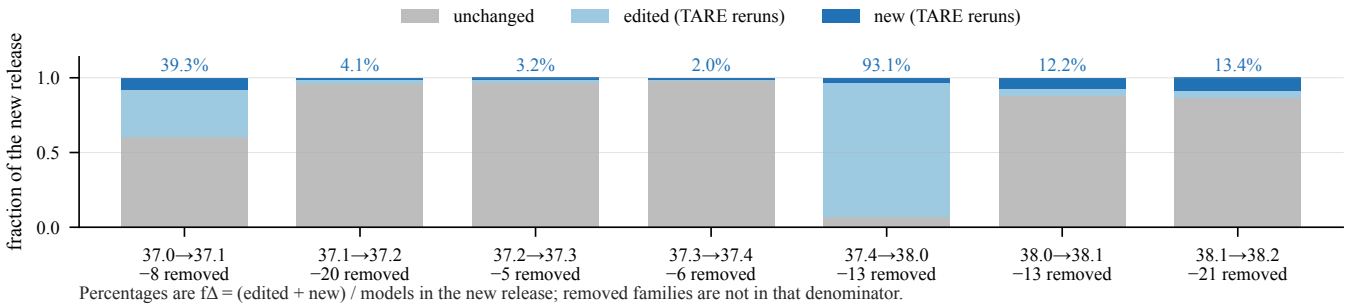


Figure 6: **Edits, not just new families, decide how much an update reruns.** Strict hashes of seven consecutive Pfam releases. Gray is unchanged. Light blue is edited and dark blue is new; both are what TARE reruns. The label is $f_\Delta = (\text{edited} + \text{new})/Z_{\text{new}}$. Pfam 37.4→38.0 reruns 93.1% of the new release: 22,965 edited and 822 new, with 1,758 unchanged, of 25,545. Pfam 38.1→38.2 reruns 13.4%: 1,378 edited and 2,674 new, with 26,082 unchanged, of 30,134, the same pair as Figure 1. The smallest change is 37.3→37.4, at 2.0%.

and Bernoulli (p=0.02, floor 20) gave 4.24× (CI 4.17–4.68); cache fill was a median 1.37× stock under the bounded policy. One pre-registered genome refused at fill and three 4-hour timeouts were rerun at 8 hours (recorded before rerun).

Infernal, with a reduced check.

Infernal reduced check (long DNA)

When one record holds >half the input residues, the sampled audit and the random pads are skipped (an audit of the whole genome *is* a stock run). What remains: the neighbor pad and the in-situ pin check. Speedups below must be read with that caveat; every known-bad control still refused.

The need for that rule showed up on a single-family removal: only 457 of Rfam 15.1's 4,227 families (10.8%) changed, yet the update took 666 s against a 328 s stock run (0.49×), exact but slower, because a bacterial genome is one DNA sequence (or a handful) and the full audit covered everything. Budgeting audit and random pads by residues as above leaves HMMER's decisions and proteome output unchanged.

With this change, Infernal updates on ten fresh genomes ran a median $S_g = 2.16\times$ faster than stock without clan competition (95% bootstrap CI 2.00–2.28; geometric mean 2.07×; range 1.41–2.37) and 2.07× with it (CI 1.97–2.23; geometric mean 2.10×; range 1.96–2.33), against cost-model predictions of 2.08× and 2.05×. These are laptop timings with three alternating runs per arm: the largest per-arm CV was 29% (flagged, not dropped, as pre-registered), and the largest per-genome prediction error was 32%.

Infernal payback is slow. Cache fill was a median 4.45× stock (payback ≈6.4 updates); Infernal is worthwhile today mainly across several Rfam releases.

6.4 Q3: Does TARE Refuse When It Should?

A failure, and what it revealed. The check's design came from a failure. When we declared DIAMOND's database-size option (`--dbsize`) as a pin, the replay on 300 proteins lost 6 of 31,377 stock lines while every other line was byte-identical, and our first check, which used only random pads, passed it. Under our pre-registered rule, that was an UNSAFE result and invalidated that check. To find the cause, we added left-out entries back in halves until each missing line returned. Every one of the six lines was restored by exactly one entry: the candidate's neighbor in the database file. Figure 7 shows the mechanism in DIAMOND's source: its first filter reads bytes across the boundary into the preceding entry. DIAMOND's results therefore depend on the order of the database. A random pad contains one particular neighbor with probability 0.049, so three random pads miss it 86% of the time, which is why the first check passed.

Table 3: Safety controls under the shipped check. Every case known to be wrong was refused, the correct case passed, and no run produced wrong output.

Control	Expected	Result
DIAMOND, 6 lines lost	refuse	REFUSE
DIAMOND, 22 lines differ	refuse	REFUSE
5% candidates dropped	refuse	REFUSE
HMMER genome (correct)	pass	pass, exact
DIAMOND, fresh sample	no wrong output	SAFE-REFUSE
BLAST+ boundary	refuse	REFUSE (R1)

Five other Swiss-Prot file orders on the same 2,000 queries changed 322 of 210,460 stock lines (173 missing, 148 extra, 1 value); 321 of those alignments sat inside the filter window of an entry boundary.

The shipped check. Adding the neighbor pad closed this gap (Table 3). On a fresh sample of 2,000 proteins, the replay differed from stock in 22 of 227,223 lines (20 missing, 2 extra), (0.0097% of lines), and the check failed on all five pads: a true SAFE-REFUSE. A second fresh DIAMOND sample was also refused. BLAST+ was refused earlier, at contract

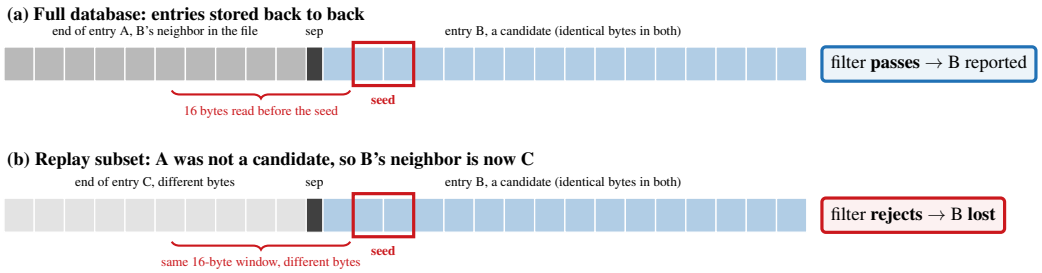


Figure 7: **Why DIAMOND cannot be replayed exactly.** DIAMOND stores database entries back to back, separated by a single byte. Its first filter examines a window of 16 bytes before and 32 after each short matching "seed" without stopping at the entry boundary (DIAMOND 2.2.5, `finger_print.h`). A seed near the start of entry B therefore reads the end of whichever entry precedes B. Removing that neighbor from the database changes the window and can drop B's hit. The neighbor pad adds A back, the file-order neighbor of every candidate; B's line then reappears, the check sees a line change, and TARE refuses (R5). Each of the six lines DIAMOND lost was restored by exactly one such neighbor.

inference: one of 15,125 aligned lines has a percent identity (column 2) that matches no normalizer, so R1 fired. That line is a different alignment, measured below. In Table 3 the correct HMMER case passed; across all HMMER updates, 6 of 63 were refused needlessly (Section 6.2).

BLAST+ is still refused at contract inference (R1): unpinned probes print a different alignment on some rows, so percent identity fits no column rule. A pinned pilot was byte-identical to stock; we have not moved the probe under the pin, so BLAST+ does not ship.

Limits and the bounded audit. Random pads are weak against non-neighbor coupling (Equation (5), Figure 8): on a synthetic scorer (1,400 trials), file-wide coupling was always refused, but hidden or distant coupling missed on 741 of 800 trials. With the earlier audit, 489 of those 800 still shipped; with the bounded audit (s=299), where at least 1% of records were affected only 3 of 400 shipped (vs. 127 of 400 earlier), within the 5% bound, and all 200 clean trials shipped. The historical bounded audit costs ≈6.4% of a stock run (vs. 2.1% earlier). Guarantee (interim): under C0/C1 with (D)–(O), (C), and (I) in Section 3, Proposition 1 gives an exact merge under the contract; refuse C2–C4 when observed; for hidden coupling among records with at least one replay candidate, wrong updates ship with probability $\leq 5\%$, assuming a valid audit oracle, when $\geq 1\%$ of that population are affected (Equation (1)). Zero-candidate records rely on pinned Z and per-family `--cut_ga`, not on this sample; the all-record audit re-timing is reported in Table 4.

A disclosed mid-run rule change (candidate-vs-pad for dual-role entries) made the check more permissive; we reran every control and every known-bad case still refused. Option search finds size pins (e.g. HMMER `-Z`) but not reporting thresholds; a missing pin is R2, never a guess.

Table 4: Audit configurations and their sampling populations. Each historical policy retains its own freeze and timing evidence. The all-record row is the locked MEASURED_PROTOCOL_v3 campaign (nine genomes, three repeats; one wall-quality FLAGGED genome excluded from the median).

Policy	Population	Median [95% CI]
Bernoulli	≥1 candidate	4.24 [4.17, 4.68]
Bounded	≥1 candidate	3.49 [3.32, 4.10]
All-record	all records	3.47 [3.30, 3.81]

Bernoulli: p=0.02, floor 20; jobs 13208152/13227819. Bounded: s=299, W13 freeze, job 13260162. All-record: s=299; n=8 unflagged (1 flagged); sensitivity 3.44×; C/B 1.58× (Table 5). With a valid oracle, each bounded policy misses ≥ 1% affected in its population with probability ≤ 5%.

6.5 Q4: Comparison with Existing Tools

General-purpose caches. On one exclusive EPYC 7542 node and one workload (Table 6), INCR's first run was 1.50× stock (95% CI 0.87–2.13), ProcessCache 14.49× (14.06–14.92), and TARE 4.20× (4.17–4.24); Riker was blocked (ptrace). Command-keyed caches re-execute on a new genome and on a new release, so a release is outside their design: they give no speedup when the reference changes. TARE's first run costs more than INCR's on this 500-protein input because fixed setup phases dominate: on the first paired run, parse took 48.9 s, press 9.7 s, the fill tool 405.5 s, selection 246.5 s, and the pin check 151.8 s, against about 240 s stock. On whole genomes under the historical Bernoulli audit, the first-run cost is a median 1.32× stock with a median break-even of 0.26 updates, and the headline update still replays only the changed part (Section 6.3). A shared-contract experiment (reuse one fill's contract across N cold genomes; $\rho = T_C/T_A$) is separate from the release-update headline. Every shared-contract output was exact under the contract (q=1.0; exactness all ok). Overhead falls as more genomes share one contract—

Table 5: Per-genome all-record S_g and C/B. A-spread = (max / min −1) on stock repeats. †FLAGGED (excluded from headline).

Genome	S_g	C/B	A%
GCF_002302335.1	3.44	1.54	4.6
GCF_009433135.1	3.28	1.71	2.9
GCF_009948655.1	3.81	1.66	40.3
GCF_022559285.1	3.28	1.57	48.0
GCF_030347155.1	3.50	1.53	10.1
GCF_049949335.1†	3.22	1.61	55.5
GCF_054388855.1	3.53	1.57	44.8
GCF_055556535.1	3.31	1.58	3.3
GCF_061234675.1	3.96	1.64	13.1

Table 6: Matched **first run** ≠ **update** cost on one exclusive EPYC 7542 node (500 proteins, Pfam-A 30,134 models, n=4 paired): $\rho = T/T_{\text{stock}}$ (mean, 95% t-interval). With n=4, INCR's interval is wide ([0.87, 2.13]); do not remember only ProcessCache's 14.49×. "New release" is what happens when the reference database changes. Riker was blocked on this cluster (`PTRACE_GET_SYSCALL_INFO`, errno 5); a laptop build measured 1.62× [1.53, 1.71] on the same protein count.

System	New release	First-run ρ
INCR [30]	reruns everything	1.50× [0.87, 2.13]
ProcessCache [26]	reruns everything	14.49× [14.06, 14.92]
Riker [7]	reruns everything	blocked (ptrace)
TARE	replays changed part	4.20× [4.17, 4.24]

$\rho(1) = 1.474$ (95% CI [1.398, 1.549], r=2 of 3 before an 8 h TIMEOUT), $\rho(5) = 1.267$ (CI [0.909, 1.908], r=3; replicate values 1.660, 1.008, 1.133), $\rho(10) = 1.167$ (CI [1.012, 1.345], r=1)—but stays above 1, because each cold genome still pays its own full search.

Approximate incremental search. iSeqSearch's incremental DIAMOND [31] on 300 proteins (Swiss-Prot 2026_01→2026_03) matched only 407 of 31,377 shared lines byte-for-byte (1.30%) and retained 133 hits to deleted sequences (Figure 9); TARE refuses and returns stock. A second locked attempt to time that refuse path failed its pre-registered control (scorer `tare_refused` false) while every timed TARE arm recorded REFUSE with stock-identical output; we state that as a limitation and do not claim an iSeqSearch speedup.

7 Discussion

Where the costs come from. A full stock verifier cannot be undercut by work stacked on top of it, and a cold genome still pays its own stock search. In shared-contract N=5 replicate 0, first-genome parse was 4,099 s vs. 6.6–7.9 s later; cache writes ranged 3.2–876 s; HMMER search was 2,609–2,944 s. **Beyond biology (designed for; measured in bio).** Exposed global normalizers (IR IDF [14, 27], mass-spec FDR [2, 13], catalog priors [10]) force the same pin-or-refuse

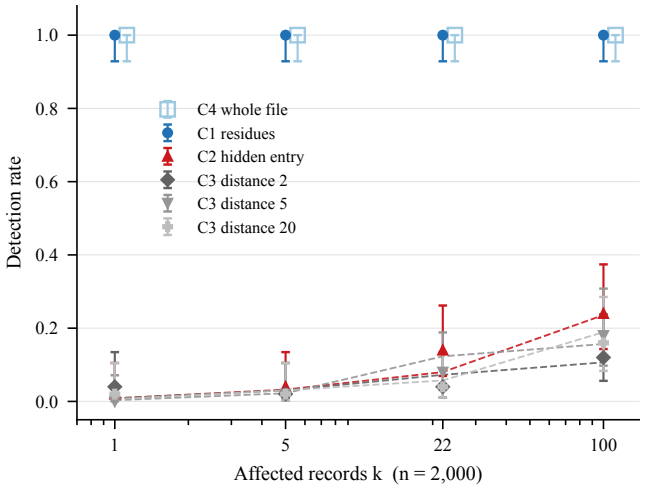


Figure 8: **The random pads miss coupling that is not a file-order neighbor.** Each point is the fraction of trials the check refused, with a Wilson 95% interval. Dashed curves are the random-pad formula (Equation (5)) for the forms it describes. File-wide coupling sits at the top of the plot; hidden and distant coupling does not. With no coupling (C0), 0 of 200 trials were refused. C4's squares are offset slightly right so both markers at 1.0 stay visible. The audit did not run in these trials.

choice. Semgrep 1.182.1 laptop shape $T(f)$=4.3+171.3·f s (97.6% rule term; not a speed claim); stock-vs-stock caught nondeterminism at `-j 32` (745 vs. 746). **Semgrep: exact, not profitable.** On the measured run (`-j 16`, r=2), updates shipped stock-identical except volatile JSON `time`, but S≈0.50 (A→B and B→C) because `--verify full` is itself a stock scan and parsing dominates cost even at f_Δ=0.0049. With HMMER (pays) and DIAMOND/BLAST+ (refused), Semgrep is a third class: exact under the contract, not profitable.

Threats to validity. *Generality:* HMMER and Infernal share lineage; DIAMOND and BLAST+ are refused from other codebases; we claim the method for programs whose coupling is in exposed settings, not that it works for most search programs (Section 6.4). SIB's `pfscanV3` (no pin) shipped 26/30 genomes byte-identical across two draws (4 below power floor); both draws are inconclusive under our rules and supporting only, so a PROSITE timing bake-off was not run. *Release choice:* Figure 6 spans 2.0%–93.1% rewrite; the headline times 38.1→38.2 (13.4%), and the chain covers 37.3→38.2 without break-even fallback (Section 6.3). *Timing:* laptop numbers are indicative; headline and matched baselines are exclusive-node alternating runs. *Check strength:* below the audit's 1% threshold, hidden coupling can ship; the all-record s=299 headline in Table 4 is the re-timed attachment of that broader sample.

Anticipated objections. A shell script reimplementing the

Table 7: Prior work on the problem of updating annotations after a database release. "Unmodified": runs the program as is. "Release-aware": reuses work when the database changes. "Identical": output equals a full run byte for byte. "Per-tool code": requires code or formulas written for each program. †TARE's pins, side files, groups, and reporting thresholds are declared as data on the command line; its core has no program-specific code.

System	Grain	Unmodified	Release-aware	Identical	Per-tool code	Limitation for this problem
Riker, ProcessCache, INCR [7, 26, 30]	command / chunk	yes	no	yes	no	a release reruns everything
Nextflow, Snakemake [11, 19]	task	yes	no	yes	no	task inputs change
iBLAST [9]	appended sequences	yes	growth only	rescaled	yes	BLAST-specific E-value formula
iSeqSearch [31]	appended sequences	yes	growth only	rescaled	yes	1.3% of lines identical; stale hits
GeStore [22]	plugin	yes	yes	rescaled	yes	administrator-written plugins
Precomputed lookups [4, 15]	query sequence	yes	no	yes	no	keyed by query, fixed release
KumQuat [25]	input stream	yes	no	yes	no	no database argument
IVM, self-adjusting [1, 3]	operator	no	yes	yes	n/a	computation must be rewritten
mpiBLAST [8]	database segment	yes	no	yes	yes	parallel search of one release
TARE	**record × entry**	**yes**	**yes**	**under contract**	**no**†	**refuses observed unsupported coupling**

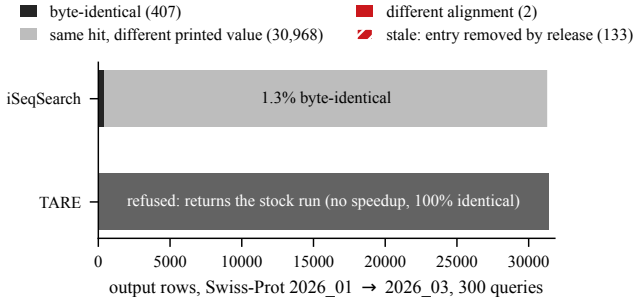


Figure 9: **Approximate versus exact.** Output of iSeqSearch's incremental DIAMOND after a Swiss-Prot update, compared line by line with a full run. Its numbers are close, but only 1.3% of lines match exactly and 133 lines report sequences the new release deleted. TARE refuses this program and returns the stock run (exact, not faster).

same batch grid is TARE without gate/check/audit, not an independent expert. On three of five locked genomes (exclusive EPYC 7542; seed 20261249), median S_g^{tare}=5.32 and S_g^{expert}=5.96 (R_g=0.866)—`inconclusive`, not the locked five. Both use a caller-declared pin. The script silently drops 68 of 227,223 DIAMOND hits; rescaling cached E-values matched only 58.0% of 44,132 HMMER hit lines.

8 Related Work

What is new. Command-keyed caches (Riker, ProcessCache, INCR, Nextflow/Snakemake) [7, 11, 19, 26, 30] rerun when input bytes change, so a release invalidates everything; they are not release-aware memoization. INCR reuses shell-stage and intermediate command effects when dependencies are unchanged [30]; Caruca mines invocation and filesystem specifications for opaque commands [17]—neither addresses coupled reference entries whose printed statistics depend on a pinned global size across database releases. iBLAST/iSeqSearch/GeStore [9, 22, 31] are release-aware but rescale or plugin-merge; we ran only iSeqSearch, whose output was not byte-identical (Figure 9). mpiBLAST [8] contributes the pin; TARE adds candidate selection, batched replay, and check-or-refuse. Black-box splitters and IVM [1, 3, 25, 29] do not split a size-coupled database argument of an unmodified binary. Probes borrow metamorphic testing [6, 24], not a new testing method.

9 Conclusion

Reference databases change every few weeks to months, and the programs that search them start from scratch every time. We showed that under the stated contract, programs whose cross-entry coupling lies in exposed settings can avoid this repeated work: pinning those settings lets an unmodified program reprint, byte for byte, the output of a full run while searching only what changed, and a check that adds back left-out entries catches coupling the pin does not cover, as it did for DIAMOND. On Pfam 38.1→38.2 under the locked all-record audit, median S_g=3.47× (Table 4). Next steps: BLAST+ with its pin, cheaper Infernal fill, and InterProScan.

Availability. Anonymized artifact (pre-registrations, addenda, results, figure scripts); URL omitted for double-blind review.

References

[1] Umut A. Acar, Guy E. Blelloch, and Robert Harper. Selective memoization. In *Proc. ACM SIGPLAN–SIGACT Symposium on Principles of Programming Languages (POPL)*, pages 14–25, 2003.

[2] Suruchi Aggarwal, Anurag Raj, Dhirendra Kumar, Debasis Dash, and Amit Kumar Yadav. FDR: The Achilles' heel of proteogenomics. *Briefings in Bioinformatics*, 23(5):bbac163, 2022.

[3] José A. Blakeley, Per-Åke Larson, and Frank Wm. Tompa. Efficiently updating materialized views. In *Proc. ACM SIGMOD International Conference on Management of Data*, pages 61–71, 1986.

[4] Carlos P. Cantalapiedra, Ana Hernández-Plaza, Ivica Letunic, Peer Bork, and Jaime Huerta-Cepas. eggNOG-mapper v2: Functional annotation, orthology assignments, and domain prediction at the metagenomic scale. *Molecular Biology and Evolution*, 38(12):5825–5829, 2021.

[5] I-Min A. Chen, Ken Chu, Krishnaveni Palaniappan, et al. The IMG/M data management and analysis system v.6.0: new tools and advanced capabilities. *Nucleic Acids Research*, 49(D1):D751–D763, 2021.

[6] T. Y. Chen, S. C. Cheung, and S. M. Yiu. Metamorphic testing: A new approach for generating next test cases. Technical Report HKUST-CS98-01, Hong Kong University of Science and Technology, 1998. arXiv:2002.12543.

[7] Charlie Curtsinger and Daniel W. Barowy. Riker: Always-correct and fast incremental builds from simple specifications. In *Proc. USENIX Annual Technical Conference (ATC)*, pages 885–898, 2022.

[8] Aaron E. Darling, Lucas Carey, and Wu chun Feng. The design, implementation, and evaluation of mpiBLAST. In *Proceedings of ClusterWorld Conference & Expo*, 2003.

[9] Sajal Dash, Sarthok Rasique Rahman, Heather M. Hines, and Wu-chun Feng. iBLAST: Incremental BLAST of new sequences via automated e-value correction. *PLoS ONE*, 16(4):e0249410, 2021.

[10] L. Delchambre, C. A. L. Bailer-Jones, I. Bellas-Velidis, R. Drimmel, D. Garabato, et al. Gaia data release 3: Apsis. III. non-stellar content and source classification. *Astronomy & Astrophysics*, 674:A31, 2023.

[11] Paolo Di Tommaso, Maria Chatzou, Evan W. Floden, Pablo Prieto Barja, Emilio Palumbo, and Cedric Notredame. Nextflow enables reproducible computational workflows. *Nature Biotechnology*, 35:316–319, 2017.

[12] Sean R. Eddy. Accelerated profile HMM searches. *PLoS Computational Biology*, 7(10):e1002195, 2011.

[13] Ari M. Frank, Matthew E. Monroe, Anuj R. Shah, Jeremy J. Carver, Nuno Bandeira, Ronald J. Moore, Gordon A. Anderson, Richard D. Smith, and Pavel A. Pevzner. Spectral archives: Extending spectral libraries to analyze both identified and unidentified spectra. *Nature Methods*, 8:587–591, 2011.

[14] Marios Hadjieleftheriou, Nick Koudas, and Divesh Srivastava. Incremental maintenance of length normalized indexes for approximate string matching. In *Proc. ACM SIGMOD International Conference on Management of Data*, pages 429–440, 2009.

[15] Philip Jones, David Binns, Hsin-Yu Chang, et al. InterProScan 5: genome-scale protein function classification. *Bioinformatics*, 30(9):1236–1240, 2014.

[16] John Jumper, Richard Evans, Alexander Pritzel, et al. Highly accurate protein structure prediction with AlphaFold. *Nature*, 596:583–589, 2021.

[17] Evangelos Lamprou, Seong-Heon Jung, Mayank Keoliya, Lukas Lazarek, Konstantinos Kallas, Michael Greenberg, and Nikos Vasilakis. Caruca: Effective and efficient specification mining for opaque software components, 2025. arXiv:2510.14279.

[18] Wenjun Li, Kathleen R. O'Neill, Daniel H. Haft, et al. RefSeq: expanding the Prokaryotic Genome Annotation Pipeline reach with protein family model curation. *Nucleic Acids Research*, 49(D1):D1020–D1028, 2021.

[19] Felix Mölder, Kim Philipp Jablonski, Brice Letcher, Michael B. Hall, Christopher H. Tomkins-Tinch, Vanessa Sochat, Jan Forster, Soohyun Lee, Sven O. Twardziok, Alexander Kanitz, Andreas Wilm, Manuel Holtgrewe, Sven Rahmann, Sven Nahnsen, and Johannes Köster. Sustainable data analysis with Snakemake. *F1000Research*, 10:33, 2021.

[20] Typhaine Paysan-Lafosse, Antonina Andreeva, Matthias Blum, Sara Rocio Chuguransky, Tiago Grego, Beatriz Lazaro Pinto, Gustavo A. Salazar, Maxwell L. Bileschi, Felipe Llinares-López, Laetitia Meng-Papaxanthos, Lucy J. Colwell, Nick V. Grishin, R. Dustin Schaeffer, Damiano Clementel, Silvio C. E. Tosatto, Erik Sonnhammer, Valerie Wood, and Alex Bateman. The Pfam protein families database: embracing AI/ML. *Nucleic Acids Research*, 53(D1):D523–D534, 2025.

[21] Typhaine Paysan-Lafosse, Matthias Blum, Sara Chuguransky, et al. InterPro in 2022. *Nucleic Acids Research*, 51(D1):D418–D427, 2023.

[22] Edvard Pedersen, Nils Peder Willassen, and Lars Ailo Bongo. Transparent incremental updates for genomics data analysis pipelines. In *Euro-Par 2013: Parallel Processing Workshops*, volume 8374 of *Lecture Notes in Computer Science*, pages 311–320. Springer, 2014.

[23] Pfam. Pfam release 38.2 release notes. https://ftp.ebi.ac.uk/pub/databases/Pfam/current_release/relnotes.txt, 2026. Accessed 2026-10-10.

[24] Sergio Segura, Gordon Fraser, Ana B. Sánchez, and Antonio Ruiz-Cortés. A survey on metamorphic testing. *IEEE Transactions on Software Engineering*, 42(9):805–824, 2016.

[25] Jiasi Shen, Martin Rinard, and Nikos Vasilakis. Automatic synthesis of parallel Unix commands and pipelines with KumQuat. In *Proc. PPoPP*, pages 431–432, 2022.

[26] Kelly Renee Shiptoski. *Reproducibility and Performance Optimizations for Unmodified Linux Programs*. PhD thesis, University of Pennsylvania, 2023.

[27] Moritz Staudinger, Florina Piroi, and Andreas Rauber. Reproducible hybrid time-travel retrieval in evolving corpora. In *Proc. ACM SIGIR-AP*, pages 203–208, 2024.

[28] The UniProt Consortium. UniProt: the universal protein knowledgebase in 2025. *Nucleic Acids Research*, 53(D1):D609–D617, 2025.

[29] Nikos Vasilakis, Konstantinos Kallas, Konstantinos Mamouras, Achilles Benetopoulos, and Lazar Cvetković. PaSh: Light-touch data-parallel shell processing. In *Proc. EuroSys*, 2021.

[30] Yizheng Xie, Evangelos Lamprou, Jerry Xia, and Nikos Vasilakis. Faster re-execution via bolt-on incrementalization. In *Proc. USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2026.

[31] Hyunwoo Yoo, Mohammadsaleh Refahi, Robi Polikar, Bahrad A. Sokhansanj, James R. Brown, and Gail L. Rosen. iSeqSearch: Incremental protein search for iBlast/iMMSeqs2/iDiamond. *PeerJ*, 13:e19171, 2025.